UNIVERSIDADE PAULISTA — UNIP

Análise e Desenvolvimento de Sistemas

Desenvolvimento Mobile

Relatórios das oito atividades práticas

Nome: [PREENCHER]
RA: [PREENCHER]
Polo de matrícula: [PREENCHER]
Local da realização da aula prática: [PREENCHER]
Data acadêmica: [PREENCHER]

Compilação documental. Consultar o status de cada atividade; evidências obrigatórias pendentes impedem a conclusão acadêmica.

Relatório 1 — Conhecendo a ferramenta Android Studio

Nome: [PREENCHER] | RA: [PREENCHER]
Polo: [PREENCHER] | Local da aula prática: [PREENCHER]
Data acadêmica: [PREENCHER]. Trabalho remoto no Codex Cloud; participação presencial não declarada.

Objetivo e resumo teórico

Criar um projeto Android básico, alterar seu texto inicial e reconhecer os arquivos usados no desenvolvimento. Android Studio é a IDE oficial para Android, reunindo editor, gerenciamento de SDK, sincronização Gradle, ferramentas de depuração e integração com dispositivos virtuais. A IDE facilita o fluxo editar, compilar, instalar, executar e inspecionar.

A Activity é o ponto de interação e recebe eventos do ciclo de vida, como onCreate. O layout XML descreve a interface de modo declarativo e é carregado com setContentView. AndroidManifest.xml declara a aplicação, as Activities e o ponto de entrada. Gradle configura plugins, versões do SDK e tarefas de compilação; o Gradle Wrapper padroniza a versão utilizada.

Atividade prática

android-nativo/app/src/main/java contém classes Java; res/layout contém as telas XML; res/values guarda textos e recursos compartilhados; AndroidManifest.xml fica em app/src/main. app/build.gradle configura o módulo; settings.gradle registra o projeto. O emulador executa uma imagem Android isolada para testar o APK. A pasta build contém saídas geradas e não substitui o código-fonte.

Implementação

```
@Override public void onCreate(Bundle state) {
 super.onCreate(state);
 setContentView(R.layout.activity_main);
}
```

Texto inicial alterado e navegação para as atividades independentes. Fontes: MainActivity.java e res/layout/activity_main.xml.

Aplicativo executado — evidência preservada



Figura 1 — Aplicativo executado em Android API 30. Fonte: execução própria anterior, PNG original preservado.

Android Studio real: interface e estrutura

O projeto existente foi aberto no Android Studio Rabbit 2 em Xvfb, com renderização por software. A sincronização Gradle terminou. A árvore expõe Manifest, Java, res/layout, res/values e Gradle; o editor mostra MainActivity.java. Não foi recriado o projeto nem refeita a atividade Android.

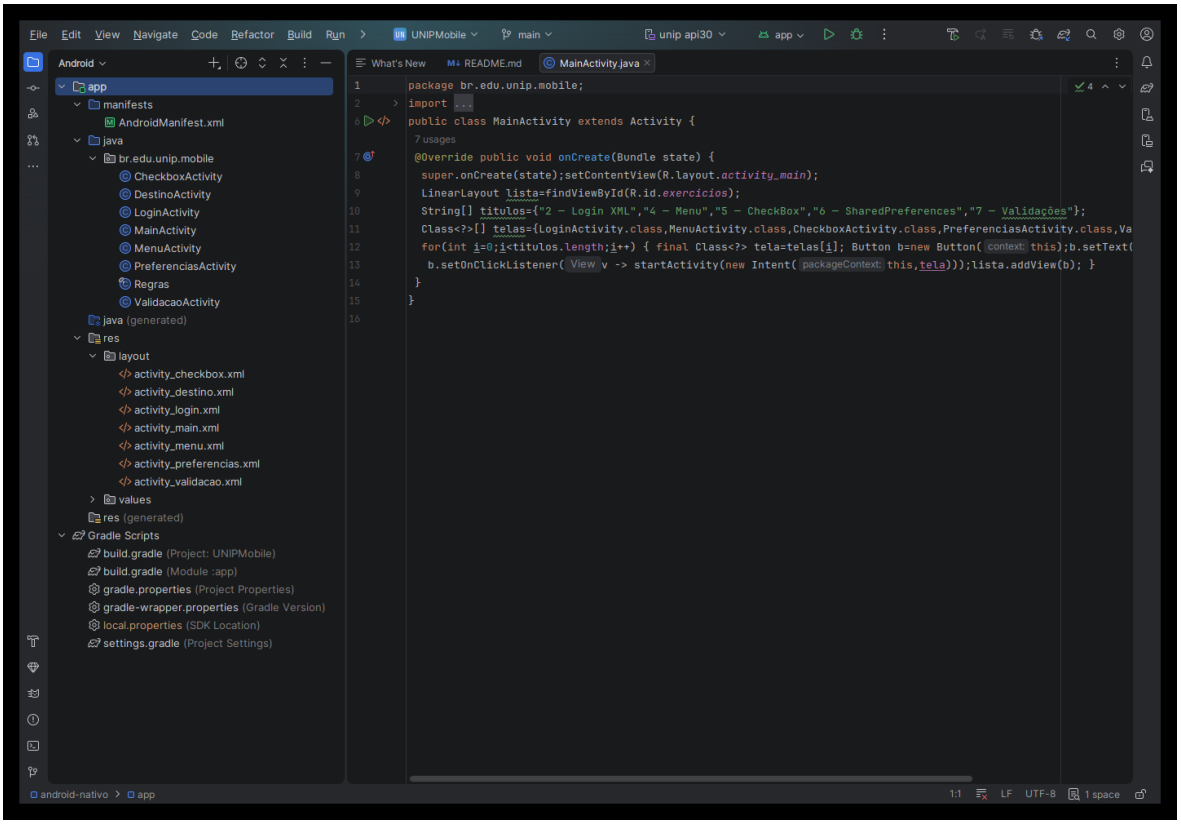


Figura 2 — Interface autêntica do Android Studio com o projeto existente, sua estrutura e código Java. Fonte: captura própria do display virtual X11.

Resultados e conclusão

A compilação e a execução anteriores permanecem válidas e documentadas nos logs e no print original. A nova captura complementa a evidência da IDE e relaciona a estrutura descrita à ferramenta real. A configuração e as capturas foram realizadas remotamente no Codex Cloud, sem declaração de participação presencial.

Referências

UNIVERSIDADE PAULISTA. Roteiros: Desenvolvimento Mobile. [S. l.]: UNIP, [s. d.]. Roteiro 1. Material didático fornecido.

UNIVERSIDADE PAULISTA. Desenvolvimento Mobile: relatório de aulas práticas. [S. l.]: UNIP, [s. d.]. Relatório 1. Material didático fornecido.

ANDROID OPEN SOURCE PROJECT. Activity: código-fonte e documentação da API. [S. l.], [s. d.]. Disponível em: https://raw.githubusercontent.com/aosp-mirror/platform_frameworks_base/master/core/java/android/app/Activity.java. Acesso em: 31 out. 2026.

ANDROID DEVELOPERS. Download Android Studio. [S. l.], [s. d.]. Disponível em: https://developer.android.com/studio. Acesso em: 31 out. 2026.

Relatório 2 — Tela de login em XML

Nome: [PREENCHER] | RA: [PREENCHER]
Polo: [PREENCHER] | Local da aula prática: [PREENCHER]
Data acadêmica: [PREENCHER]. Trabalho remoto no Codex Cloud; participação presencial não declarada.

Objetivo e resumo teórico

Construir uma tela com título, e-mail, senha e botão Entrar. XML é uma linguagem de marcação usada para declarar componentes e atributos. Separar interface e lógica permite ajustar tamanhos, espaçamentos e textos sem misturar essas decisões com o tratamento do clique.

TextView apresenta o título e o resultado; EditText coleta e-mail e senha, usando tipos de entrada adequados; Button dispara uma ação; LinearLayout dispõe os filhos verticalmente. Um ScrollView externo mantém acesso em telas menores. Os textos de interface do layout estão em res/values/strings.xml, com margens e áreas de toque mínimas.

Atividade prática

A Activity carrega activity_login.xml, obtém as referências com findViewById e registra um OnClickListener. Campos vazios geram orientação. Quando preenchidos, o resultado mostra uma saudação e identifica a demonstração como local. A senha é limpa após o clique bem-sucedido. Não existe servidor, sessão ou autenticação.

Implementação e códigos

```
<LinearLayout android:orientation="vertical" ...>
 <TextView android:text="@string/texto_1" ... />
 <EditText android:id="@+id/email"
   android:inputType="textEmailAddress" ... />
 <EditText android:id="@+id/senha"
   android:inputType="textPassword" ... />
 <Button android:id="@+id/entrar" ... />
</LinearLayout>
```

As reticências identificam um recorte explicativo, não um XML executável. O arquivo integral anexo contém os atributos de tamanho, espaçamento e recursos e é a fonte usada na compilação.

Fontes integrais: android-nativo/app/src/main/java/br/edu/unip/mobile/LoginActivity.java e android-nativo/app/src/main/res/layout/activity_login.xml. Regras adicionais: Regras.java. Todos os arquivos são anexos de código no repositório.

Testes e resultados reais

Compilação da base aprovada (testes/android-base-build.log); compilação das seis telas e Android Lint aprovados (testes/android-final-build.log). Quinze verificações de lógica Java passaram (testes/java-regras.log). Esses resultados não substituem a execução e os testes de interface. A situação específica da execução está registrada na seção de evidências.

Conclusão

A separação entre XML e Java facilita a manutenção. Uma interface de login é apenas a apresentação: a validação visual e a saudação não comprovam identidade. Autenticação de produção exigiria serviço confiável, proteção de comunicação e gerenciamento de sessão.

Testes funcionais e evidências reais

3 verificações aprovadas via ADB e UI Automator. Plataforma: Emulador Android API 30 x86_64, TCG sem KVM. Resultado conferido pela árvore da interface antes de cada captura; PNG original preservado. Registro: testes/android-funcional-02.json.



Figura 1 — Login XML com título, e-mail, senha e botão. Fonte: execução própria.



Figura 2 — Saudação da interação demonstrativa. Fonte: execução própria.

Execução e capturas concluídas nesta plataforma. Identificação acadêmica e local da aula prática permanecem a preencher; não foi declarada presença em laboratório.

Referências

UNIVERSIDADE PAULISTA. Roteiros: Desenvolvimento Mobile. [S. l.]: UNIP, [s. d.]. Roteiro 2. Material didático fornecido.

UNIVERSIDADE PAULISTA. Desenvolvimento Mobile: relatório de aulas práticas. [S. l.]: UNIP, [s. d.]. Relatório 2. Material didático fornecido.

ANDROID OPEN SOURCE PROJECT. EditText: código-fonte e documentação da API. [S. l.], [s. d.]. Disponível em: https://raw.githubusercontent.com/aosp-mirror/platform_frameworks_base/master/core/java/android/widget/EditText.java. Acesso em: 31 out. 2026.

Relatório 3 — Tela de login com Flutter e DartPad

Nome: [PREENCHER] | RA: [PREENCHER]
Polo: [PREENCHER] | Local da aula prática: [PREENCHER]
Data acadêmica: [PREENCHER]. Desenvolvimento remoto no Codex Cloud; sem declaração de participação presencial.

Objetivo e fundamentos

Construir uma tela de login demonstrativa com e-mail, senha, botão e mensagem de boas-vindas. Flutter é um framework de interfaces multiplataforma: descreve a interface como uma árvore de widgets e desenha seus componentes com um mecanismo de renderização próprio. Dart é a linguagem utilizada, com tipos, classes e compilação para diferentes plataformas.

Um widget é uma descrição imutável de uma parte da interface. MaterialApp configura tema e navegação; Scaffold fornece a estrutura da tela e AppBar; TextField recebe a entrada; ElevatedButton dispara a ação. StatefulWidget mantém o estado da mensagem; setState solicita a atualização visual após validar os campos.

Implementação e organização

O projeto está em flutter/relatorio03-login. O arquivo lib/main.dart é completo e pode ser copiado para o modo Flutter do DartPad, sem pacotes externos. A tela usa largura máxima de 420 pixels, rolagem, rótulos nos campos e tema consistente. A senha fica oculta e seu controlador é limpo após o sucesso; os dois controladores são liberados em dispose.

A ação Login verifica campos vazios e o formato básico de e-mail. Com dados preenchidos, exibe Bem-vindo(a), seguido do e-mail informado. Não envia requisições, verifica identidade ou cria sessão. Esse comportamento não representa autenticação em produção.

Trechos centrais e código integral

```
void main() => runApp(const LoginApp());
// MaterialApp -> Scaffold -> TextField / ElevatedButton
ElevatedButton(onPressed: entrar, child: const Text('Login'))
// Em entrar():
// vazio -> Preencha e-mail e senha.
// formato inválido -> Informe um e-mail válido.
// válido -> Bem-vindo(a), seguido do e-mail; senha.clear().
```

Código integral: flutter/relatorio03-login/lib/main.dart. O anexo em arquivo preserva a implementação completa sem ultrapassar o limite de páginas do relatório.

Testes e resultados reais

Flutter 3.35.7 e Dart 3.9.2 instalados com SHA-256 oficial. flutter pub get, flutter analyze, flutter test e flutter build web aprovados. A suíte executou 1 teste(s) de widget/lógica. O aplicativo compilado também foi servido localmente e testado em Chromium: 5 verificações de interface aprovadas. Logs e registros: testes/flutter-validacao-03.json e flutter-funcional-03.json.

Procedimento de reprodução: flutter pub get; flutter analyze; flutter test; flutter build web --no-web-resources-cdn; flutter run -d chrome. Depois de executar, preencher e-mail e uma senha fictícia, acionar Login e conferir a mensagem. Capturar a tela inicial e a mensagem de boas-vindas.

Conclusão — comparação entre plataformas

No Android nativo, Activity Java coordena o ciclo de vida e o XML declara componentes da plataforma Android. Flutter usa Dart e widgets para compartilhar a interface entre Android, web e outras plataformas. O compartilhamento de código e a atualização rápida favorecem prototipagem; APIs específicas podem exigir integração nativa. O funcionamento foi confirmado por testes de widget e execução real no navegador; a mesma fonte mantém compatibilidade com a proposta do DartPad.

Evidências autênticas e resultados

Aplicativo Flutter real compilado e executado em Chromium. 5 verificações de interface aprovadas; apenas dados fictícios, sem armazenamento ou logs de senhas. Os PNG originais permanecem em evidencias/relatorio03.

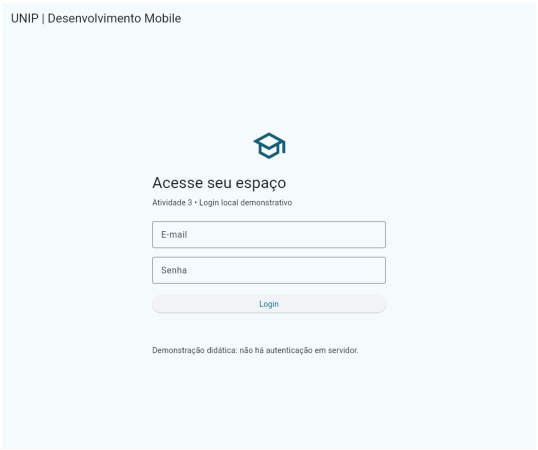


Figura 1 — Login Flutter executado no navegador.
Fonte: execução própria em Chromium.

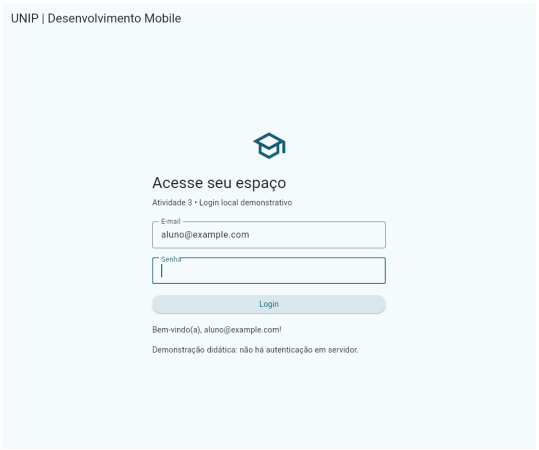


Figura 2 — Mensagem real de boas-vindas após Login.
Fonte: execução própria em Chromium.

Requisitos técnicos e capturas concluídos. Identificação acadêmica permanece a preencher pelo estudante.

Referências

UNIVERSIDADE PAULISTA. Roteiros: Desenvolvimento Mobile. [S. l.]: UNIP, [s. d.]. Roteiro 3. Material didático fornecido.

UNIVERSIDADE PAULISTA. Desenvolvimento Mobile: relatório de aulas práticas. [S. l.]: UNIP, [s. d.]. Relatório 3. Material didático fornecido.

FLUTTER. Framework library: widgets e ciclo de vida. [S. l.], [s. d.]. Disponível em: https://raw.githubusercontent.com/flutter/flutter/stable/packages/flutter/lib/src/widgets/framework.dart. Acesso em: 31 out. 2026.

DART. Dart overview. [S. l.], [s. d.]. Disponível em: https://raw.githubusercontent.com/dart-lang/site-www/main/src/content/overview.md. Acesso em: 31 out. 2026.

Relatório 4 — Menu simples em XML

Nome: [PREENCHER] | RA: [PREENCHER]
Polo: [PREENCHER] | Local da aula prática: [PREENCHER]
Data acadêmica: [PREENCHER]. Trabalho remoto no Codex Cloud; participação presencial não declarada.

Objetivo e resumo teórico

Criar um menu com Cadastro, Consultar, Configurações e Sair. Navegação representa os caminhos entre telas e deve manter rótulos claros, retorno previsível e ações coerentes. Um menu inicial organiza destinos frequentes e reduz a necessidade de descobrir funções por tentativa.

O menu é um LinearLayout vertical com Button e ícones nativos drawableStart. Ícones acompanham palavras, preservando o significado mesmo quando o desenho não é reconhecido. Espaçamentos e áreas mínimas de toque tornam a seleção mais previsível.

Atividade prática

Cadastro e Consultar abrem DestinoActivity por Intent explícita, com título e mensagem próprios, e o botão Voltar encerra apenas o destino. Configurações abre um diálogo que descreve o modo local. Sair executa finishAffinity para encerrar as Activities da tarefa. As telas são demonstrações; não há cadastro ou consulta em banco de dados.

Implementação e códigos

```
<Button android:id="@+id/cadastro"
 android:drawableStart="@android:drawable/ic_menu_add" ... />
<Button android:id="@+id/consultar" ... />
<Button android:id="@+id/configuracoes" ... />
<Button android:id="@+id/sair" ... />
```

O XML completo está no arquivo indicado. Cada opção possui um listener; DestinoActivity.java e activity_destino.xml implementam o retorno. Nenhum botão é meramente decorativo.

Fontes integrais: android-nativo/app/src/main/java/br/edu/unip/mobile/MenuActivity.java e android-nativo/app/src/main/res/layout/activity_menu.xml. Regras adicionais: Regras.java. Todos os arquivos são anexos de código no repositório.

Testes e resultados reais

Compilação da base aprovada (testes/android-base-build.log); compilação das seis telas e Android Lint aprovados (testes/android-final-build.log). Quinze verificações de lógica Java passaram (testes/java-regras.log). Esses resultados não substituem a execução e os testes de interface. A situação específica da execução está registrada na seção de evidências.

Conclusão

A navegação consistente ajuda o usuário a compreender onde está e como retornar. Nesta solução, os destinos e mensagens são simples, porém cada opção tem efeito verificável. Sair encerra a tarefa em vez de simular um logout de servidor.

Testes funcionais e evidências reais

5 verificações aprovadas via ADB e UI Automator. Plataforma: Emulador Android API 30 x86_64, TCG sem KVM. Resultado conferido pela árvore da interface antes de cada captura; PNG original preservado. Registro: testes/android-funcional-04.json.



Figura 1 — Menu com quatro opções e ícones. Fonte: execução própria.

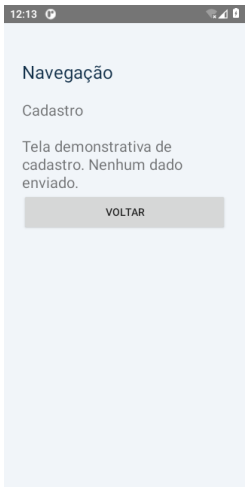


Figura 2 — Destino Cadastro. Fonte: execução própria.

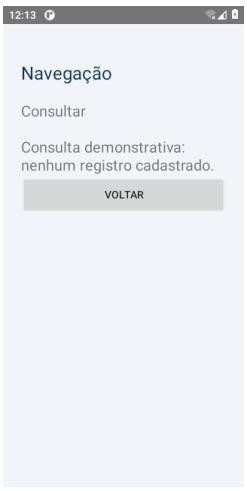


Figura 3 — Destino Consultar. Fonte: execução própria.

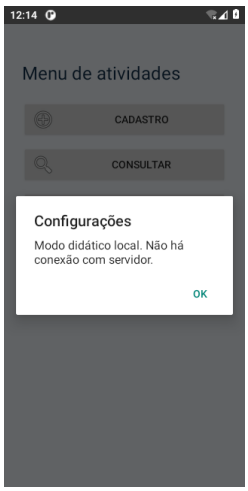


Figura 4 — Diálogo Configurações. Fonte: execução própria.

Execução e capturas concluídas nesta plataforma. Identificação acadêmica e local da aula prática permanecem a preencher; não foi declarada presença em laboratório.

Referências

UNIVERSIDADE PAULISTA. Roteiros: Desenvolvimento Mobile. [S. l.]: UNIP, [s. d.]. Roteiro 4. Material didático fornecido.

UNIVERSIDADE PAULISTA. Desenvolvimento Mobile: relatório de aulas práticas. [S. l.]: UNIP, [s. d.]. Relatório 4. Material didático fornecido.

ANDROID OPEN SOURCE PROJECT. Activity: código-fonte e documentação da API. [S. l.], [s. d.]. Disponível em: https://raw.githubusercontent.com/aosp-mirror/platform_frameworks_base/master/core/java/android/app/Activity.java. Acesso em: 31 out. 2026.

Relatório 5 — Aplicativo utilizando CheckBox

Nome: [PREENCHER] | RA: [PREENCHER]
Polo: [PREENCHER] | Local da aula prática: [PREENCHER]
Data acadêmica: [PREENCHER]. Trabalho remoto no Codex Cloud; participação presencial não declarada.

Objetivo e resumo teórico

Capturar opções independentes para a pergunta “Quais tecnologias você conhece?”. CheckBox representa uma escolha booleana e permite múltiplas seleções, ao contrário de um conjunto de RadioButtons para escolha única. É aplicável a preferências, interesses e itens de formulários.

A tela apresenta Java, Kotlin, Flutter e Python, seguidos do botão Verificar Resposta. Os componentes ficam alinhados em uma coluna e cada opção tem área de toque de pelo menos 48 dp. A seleção só é avaliada no clique, tornando o fluxo explícito para o usuário.

Atividade prática

O listener percorre os quatro IDs e consulta isChecked. Para cada opção marcada, adiciona getText à lista. Regras.selecoes produz uma mensagem específica para lista vazia ou concatena os nomes selecionados em ordem. Um TextView apresenta o resultado de forma persistente na tela.

Implementação e códigos

```
ArrayList<String> escolhas = new ArrayList<>();
// Para cada CheckBox do layout:
if (check.isChecked())
 escolhas.add(check.getText().toString());
resultado.setText(Regras.selecoes(escolhas));
```

O código Java integral inclui a associação dos IDs e o listener. O XML integral contém quatro CheckBoxes e o botão, preservados como arquivos separados para leitura e reprodução.

Fontes integrais: android-nativo/app/src/main/java/br/edu/unip/mobile/CheckboxActivity.java e android-nativo/app/src/main/res/layout/activity_checkbox.xml. Regras adicionais: Regras.java. Todos os arquivos são anexos de código no repositório.

Testes e resultados reais

Compilação da base aprovada (testes/android-base-build.log); compilação das seis telas e Android Lint aprovados (testes/android-final-build.log). Quinze verificações de lógica Java passaram (testes/java-regras.log). Esses resultados não substituem a execução e os testes de interface. A situação específica da execução está registrada na seção de evidências.

Conclusão

CheckBox permite formulários com escolhas simultâneas sem obrigar o usuário a selecionar um item. Tratar explicitamente a lista vazia evita mensagem ambígua. A lista produzida pode ser utilizada em outras etapas de um formulário, com validações próprias do domínio.

Testes funcionais e evidências reais

3 verificações aprovadas via ADB e UI Automator. Plataforma: Emulador Android API 30 x86_64, TCG sem KVM. Resultado conferido pela árvore da interface antes de cada captura; PNG original preservado. Registro: testes/android-funcional-05.json.

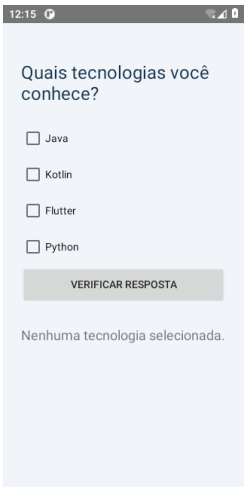


Figura 1 — Resultado sem selecionar tecnologias. Fonte: execução própria.

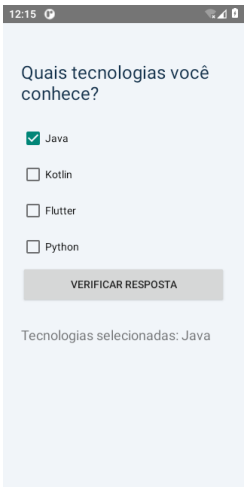


Figura 2 — Resultado com uma tecnologia selecionada. Fonte: execução própria.

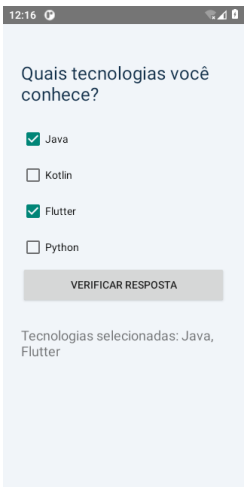


Figura 3 — Resultado com múltiplas tecnologias. Fonte: execução própria.

Execução e capturas concluídas nesta plataforma. Identificação acadêmica e local da aula prática permanecem a preencher; não foi declarada presença em laboratório.

Referências

UNIVERSIDADE PAULISTA. Roteiros: Desenvolvimento Mobile. [S. l.]: UNIP, [s. d.]. Roteiro 5. Material didático fornecido.

UNIVERSIDADE PAULISTA. Desenvolvimento Mobile: relatório de aulas práticas. [S. l.]: UNIP, [s. d.]. Relatório 5. Material didático fornecido.

ANDROID OPEN SOURCE PROJECT. CheckBox: código-fonte e documentação da API. [S. l.], [s. d.]. Disponível em: https://raw.githubusercontent.com/aosp-mirror/platform_frameworks_base/master/core/java/android/widget/CheckBox.java. Acesso em: 31 out. 2026.

Relatório 6 — Armazenamento com SharedPreferences

Nome: [PREENCHER] | RA: [PREENCHER]
Polo: [PREENCHER] | Local da aula prática: [PREENCHER]
Data acadêmica: [PREENCHER]. Trabalho remoto no Codex Cloud; participação presencial não declarada.

Objetivo e resumo teórico

Salvar e recuperar o nome do usuário no armazenamento local. Dados mantidos apenas em variáveis desaparecem quando o processo termina; dados persistidos podem sobreviver a esse encerramento. SharedPreferences armazena pares chave-valor simples em um arquivo privado da aplicação.

getSharedPreferences("perfil", MODE_PRIVATE) seleciona o arquivo; getString("nome", "") lê o valor anterior ou um padrão. Um Editor recebe putString e commit confirma a escrita. Foi escolhido commit para verificar o retorno booleano nesta demonstração com um único valor pequeno. Em fluxos maiores, escrita síncrona na thread da interface deve ser evitada.

Atividade prática

PreferenciasActivity recupera o nome em onCreate, preenche o EditText e informa “Nome recuperado”. O botão Salvar nome rejeita entrada vazia e informa sucesso ou falha. Encerrar a Activity sem matar o processo não é prova suficiente de persistência: o teste previsto força o encerramento do aplicativo e o reabre.

Implementação e códigos

```
SharedPreferences prefs = getSharedPreferences("perfil", MODE_PRIVATE);
String salvo = prefs.getString("nome", "");
boolean ok = prefs.edit().putString("nome", valor).commit();
```

O arquivo não contém senhas. MODE_PRIVATE restringe o acesso normal ao sandbox do aplicativo, mas não representa criptografia. Limpar os dados ou desinstalar o app remove esse armazenamento; fechar o processo normalmente não o remove.

Fontes integrais:
android-nativo/app/src/main/java/br/edu/unip/mobile/PreferenciasActivity.java e android-nativo/app/src/main/res/layout/activity_preferencias.xml. Regras adicionais: Regras.java. Todos os arquivos são anexos de código no repositório.

Testes e resultados reais

Compilação da base aprovada (testes/android-base-build.log); compilação das seis telas e Android Lint aprovados (testes/android-final-build.log). Quinze verificações de lógica Java passaram (testes/java-regras.log). Esses resultados não substituem a execução e os testes de interface. A situação específica da execução está registrada na seção de evidências.

Conclusão

O armazenamento local melhora a continuidade da experiência ao recuperar preferências sem exigir redigitação. SharedPreferences atende valores simples desta atividade. Dados sensíveis exigem análise adicional e os mecanismos de armazenamento devem ser escolhidos conforme o caso, sem confundir persistência com segurança.

Testes funcionais e evidências reais

3 verificações aprovadas via ADB e UI Automator. Plataforma: Emulador Android API 30 x86_64, TCG sem KVM. Resultado conferido pela árvore da interface antes de cada captura; PNG original preservado. Registro: testes/android-funcional-06.json.

Persistência confirmada: nome fictício salvo, processo encerrado com am force-stop e Activity reaberta; o texto Nome recuperado foi conferido. Os dados da aplicação não foram apagados entre essas etapas.



Figura 1 — Nome salvo no armazenamento local. Fonte: execução própria.

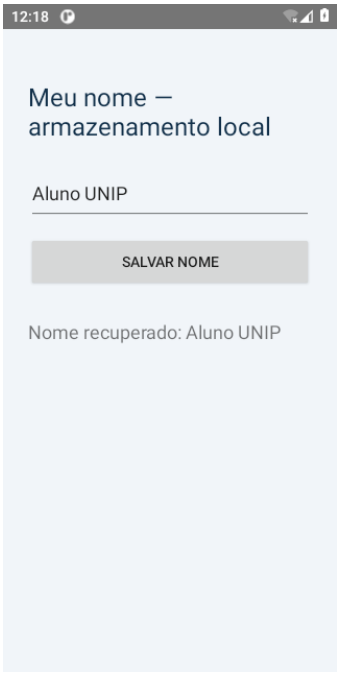


Figura 2 — Nome recuperado após forçar parada e reabrir. Fonte: execução própria.

Execução e capturas concluídas nesta plataforma. Identificação acadêmica e local da aula prática permanecem a preencher; não foi declarada presença em laboratório.

Referências

UNIVERSIDADE PAULISTA. Roteiros: Desenvolvimento Mobile. [S. l.]: UNIP, [s. d.]. Roteiro 6. Material didático fornecido.

UNIVERSIDADE PAULISTA. Desenvolvimento Mobile: relatório de aulas práticas. [S. l.]: UNIP, [s. d.]. Relatório 6. Material didático fornecido.

ANDROID OPEN SOURCE PROJECT. SharedPreferences: código-fonte e documentação da API. [S. l.], [s. d.]. Disponível em: https://raw.githubusercontent.com/aosp-mirror/platform_frameworks_base/master/core/java/android/content/SharedPreferences.java. Acesso em: 31 out. 2026.

Relatório 7 — Validações e conceitos de segurança

Nome: [PREENCHER] | RA: [PREENCHER]
Polo: [PREENCHER] | Local da aula prática: [PREENCHER]
Data acadêmica: [PREENCHER]. Trabalho remoto no Codex Cloud; participação presencial não declarada.

Objetivo e resumo teórico

Validar campos obrigatórios, formato de e-mail e senha com menos de seis caracteres. Validação de entrada verifica conformidade com regras; autenticação estabelece a identidade de quem solicita acesso. Segurança também envolve confidencialidade, integridade, disponibilidade e controle de acesso.

A tela usa EditText com rótulos, senha oculta, botão Validar e resultado textual. A regra de e-mail verifica formato básico, sem garantir que a caixa postal exista. A regra de senha adota o limite mínimo de seis caracteres do roteiro, contado por pontos de código; esse mínimo acadêmico não é uma política recomendada para produção.

Atividade prática

Regras.erroEmail distingue obrigatório e formato inválido. Regras.erroSenha distingue vazio e menos de seis caracteres. A Activity limpa erros anteriores, aplica setError em cada campo e exibe mensagens específicas no resultado. Com entradas válidas, informa apenas que os dados passaram pelas regras locais, sem autenticação em servidor.

Implementação e códigos

```
if (email.trim().isEmpty()) return "E-mail obrigatório.";
if (senha.isEmpty()) return "Senha obrigatória.";
if (senha.codePointCount(0, senha.length()) < 6)
 return "Senha deve ter pelo menos 6 caracteres.";
```

A lógica compartilhada completa está em Regras.java. Não há gravação nem logs de senha. Testes devem usar apenas exemplos fictícios. Sistemas reais precisam repetir validações no servidor e proteger credenciais, comunicação e sessão; a validação local pode ser contornada.

Fontes integrais: android-nativo/app/src/main/java/br/edu/unip/mobile/ValidacaoActivity.java e android-nativo/app/src/main/res/layout/activity_validacao.xml. Regras adicionais: Regras.java. Todos os arquivos são anexos de código no repositório.

Testes e resultados reais

Compilação da base aprovada (testes/android-base-build.log); compilação das seis telas e Android Lint aprovados (testes/android-final-build.log). Quinze verificações de lógica Java passaram (testes/java-regras.log). Esses resultados não substituem a execução e os testes de interface. A situação específica da execução está registrada na seção de evidências.

Conclusão

A validação fornece retorno imediato e reduz erros de preenchimento. Ela complementa, mas não substitui, autenticação e autorização. As mensagens específicas tornam o comportamento verificável; o sucesso local não é anunciado como acesso seguro ou identidade comprovada.

Testes funcionais e evidências reais

5 verificações aprovadas via ADB e UI Automator. Plataforma: Emulador Android API 30 x86_64, TCG sem KVM. Resultado conferido pela árvore da interface antes de cada captura; PNG original preservado. Registro: testes/android-funcional-07.json.

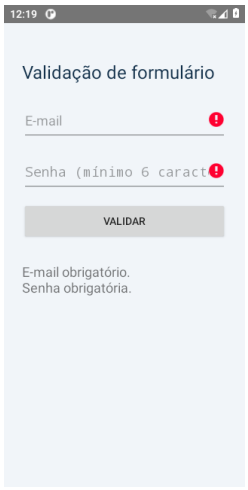


Figura 1 — Erros dos campos obrigatórios. Fonte: execução própria.

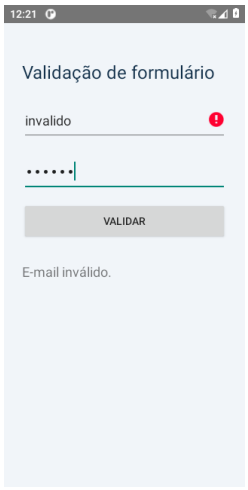


Figura 2 — Erro de e-mail inválido. Fonte: execução própria.

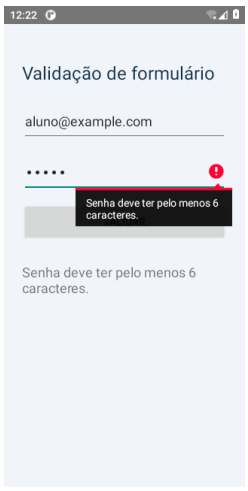


Figura 3 — Erro de senha inferior a seis caracteres. Fonte: execução própria.

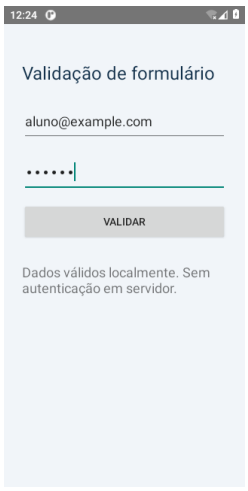


Figura 4 — Entradas aceitas pela validação local. Fonte: execução própria.

Execução e capturas concluídas nesta plataforma. Identificação acadêmica e local da aula prática permanecem a preencher; não foi declarada presença em laboratório.

Referências

UNIVERSIDADE PAULISTA. Roteiros: Desenvolvimento Mobile. [S. l.]: UNIP, [s. d.]. Roteiro 7. Material didático fornecido.

UNIVERSIDADE PAULISTA. Desenvolvimento Mobile: relatório de aulas práticas. [S. l.]: UNIP, [s. d.]. Relatório 7. Material didático fornecido.

ANDROID OPEN SOURCE PROJECT. EditText: código-fonte e documentação da API. [S. l.], [s. d.]. Disponível em: https://raw.githubusercontent.com/aosp-mirror/platform_frameworks_base/master/core/java/android/widget/EditText.java. Acesso em: 31 out. 2026.

OWASP FOUNDATION. Authentication cheat sheet. [S. l.], [s. d.]. Disponível em: https://raw.githubusercontent.com/OWASP/CheatSheetSeries/master/cheatsheets/Authentication_Cheat_Sheet.md. Acesso em: 31 out. 2026.

Relatório 8 — Verificador de senha com Flutter

Nome: [PREENCHER] | RA: [PREENCHER]
Polo: [PREENCHER] | Local da aula prática: [PREENCHER]
Data acadêmica: [PREENCHER]. Desenvolvimento remoto no Codex Cloud; sem declaração de participação presencial.

Objetivo e fundamentos

Criar uma interface Flutter que analise localmente uma senha e atualize sua classificação. Validação de dados aplica regras explícitas às entradas. Uma senha longa e única tende a resistir melhor a tentativas de adivinhação, mas uma regra de tamanho e classes de caracteres não mede sozinha a segurança efetiva.

Flutter organiza a tela em widgets. MaterialApp define o tema; Scaffold estrutura a página; TextField oculta o texto e usa onChanged; setState atualiza um Text com rótulo e cor. O resultado é comunicado por texto, além da cor, e Semantics com liveRegion favorece a indicação da mudança a tecnologias assistivas.

Critérios adotados

O roteiro não define limites numéricos. Nesta implementação, Forte exige pelo menos 12 caracteres e quatro classes; Média exige pelo menos 8 caracteres e duas classes, desde que não seja Forte; os demais casos são Fraca. As classes são minúsculas ASCII, maiúsculas ASCII, dígitos e símbolos diferentes de espaço. A contagem usa runes (pontos de código Unicode), não unidades UTF-16. Campo vazio solicita digitação, sem classificação.

A heurística é didática: não detecta padrões previsíveis, vazamentos, reutilização ou entropia; letras acentuadas entram na classe de símbolos desta regra simplificada. Não há armazenamento nem logs do texto digitado. Usar apenas entradas fictícias nos testes.

Lógica e código integral

```
if (tamanho >= 12 && classes == 4) return 'Senha Forte';
if (tamanho >= 8 && classes >= 2) return 'Senha Média';
return 'Senha Fraca';
```

Código integral: flutter/relatorio08-senhas/lib/main.dart. A função classificarSenha é independente da tela. O callback onChanged calcula o resultado e setState atualiza a interface; o campo vazio restaura a instrução inicial.

Testes e resultados reais

Flutter 3.35.7 e Dart 3.9.2 instalados com SHA-256 oficial. flutter pub get, flutter analyze, flutter test e flutter build web aprovados. A suíte executou 2 teste(s) de widget/lógica. O aplicativo compilado também foi servido localmente e testado em Chromium: 4 verificações de interface aprovadas. Logs e registros: testes/flutter-validacao-08.json e flutter-funcional-08.json.

Para reproduzir no SDK instalado: executar flutter pub get, flutter analyze e flutter test. Compilar para web e abrir no navegador. Usar três entradas exclusivamente fictícias que atendam às regras e registrar cada rótulo. Verificar também retorno à instrução inicial ao apagar o campo. Não registrar senhas reais em logs, capturas ou relatórios.

Conclusão e aplicação prática

A implementação explicita as regras para permitir testes reproduzíveis. Em sistemas reais, um indicador deve orientar sem prometer segurança absoluta. São necessários senhas longas e exclusivas, proteção de transporte, armazenamento no servidor com algoritmo apropriado de derivação de senha e autenticação multifator. A execução no navegador confirmou os três níveis e o retorno à instrução inicial ao limpar o campo.

Evidências autênticas e resultados

Aplicativo Flutter real compilado e executado em Chromium. 4 verificações de interface aprovadas; apenas dados fictícios, sem armazenamento ou logs de senhas. Os PNG originais permanecem em evidencias/relatorio08.



Figura 1 — Classificação Senha Fraca. Fonte: execução própria em Chromium.



Figura 2 — Classificação Senha Média. Fonte: execução própria em Chromium.



Figura 3 — Classificação Senha Forte. Fonte: execução própria em Chromium.

Requisitos técnicos e capturas concluídos. Identificação acadêmica permanece a preencher pelo estudante.

Referências

UNIVERSIDADE PAULISTA. Roteiros: Desenvolvimento Mobile. [S. l.]: UNIP, [s. d.]. Roteiro 8. Material didático fornecido.

UNIVERSIDADE PAULISTA. Desenvolvimento Mobile: relatório de aulas práticas. [S. l.]: UNIP, [s. d.]. Relatório 8. Material didático fornecido.

FLUTTER. Framework library: widgets e ciclo de vida. [S. l.], [s. d.]. Disponível em: https://raw.githubusercontent.com/flutter/flutter/stable/packages/flutter/lib/src/widgets/framework.dart. Acesso em: 31 out. 2026.

OWASP FOUNDATION. Authentication cheat sheet. [S. l.], [s. d.]. Disponível em: https://raw.githubusercontent.com/OWASP/CheatSheetSeries/master/cheatsheets/Authentication_Cheat_Sheet.md. Acesso em: 31 out. 2026.